SPACIUM

Système de réservation de salle

Projet préparé par le
Groupe 3

Table des Matières

Table des Matières 2
1. Introduction 4
1.1 Présentation du Projet 4
1.2 Objectifs 4
1.3 Périmètre Fonctionnel 4
2. Architecture Technique 4
2.1 Vue d'ensemble 4
2.2 Pile Technologique 4
2.3 Pattern MVVM 5
2.4 Injection de Dépendances 5
3. Structure du Projet 5
3.1 Organisation des Dossiers 5
4. Base de Données 6
4.1 Technologie 6
4.2 Schéma de la Base de Données 6
4.3 Statuts des Réservations 6
4.4 Vues SQL 7
5. Système d'Authentification 7
5.1 Mécanisme 7
5.2 Rôles et Permissions 7
5.3 Comptes par Défaut 7
6. Fonctionnalités Détaillées 8
6.1 Réservation de Salle 8
Processus de réservation 8
6.2 Gestion des Réservations 8
6.3 Gestion des Salles (Gestionnaire) 8
6.4 Gestion des Utilisateurs (Gestionnaire) 8
7. Interface Utilisateur 9
7.1 Design System 9
7.2 Navigation 9
7.3 Convertisseurs XAML 9
7.4 Fenêtres et Dialogues 9
8. Données Initiales 10
8.1 Salles Pré-configurées 10

8.2 Créneaux Horaires 10
9. Installation et Déploiement 10
9.1 Pré-requis 10
9.2 Initialisation de la Base de Données 10
9.3 Compilation 11
10. Dépendances NuGet 11
11. Points Clés de l'Architecture 11
11.1 Gestion des Statuts 11
11.2 Détection de Conflits 11
11.3 Validation 11
11.4 Séparation des Responsabilités 12
12. Glossaire 12

1. Introduction

1.1 Présentation du Projet

Spacium est une application de bureau Windows développée pour faciliter la gestion et la réservation de salles au sein d'établissements scolaires. Elle propose une interface moderne et intuitive permettant aux utilisateurs de consulter les salles disponibles, d'effectuer des réservations et de gérer leurs plannings de manière efficace.

Le gestionnaire dispose quant à lui d'un panneau d'administration complet pour superviser l'ensemble des réservations, gérer les salles, les équipements et les utilisateurs.

1.2 Objectifs

- Permettre la réservation rapide et intuitive de salles
- Gérer les conflits de réservation automatiquement
- Offrir une vue en temps réel des statuts de réservation
- Fournir un panneau d'administration complet pour les gestionnaires
- Gérer les équipements par salle
- Assurer la sécurité avec un système d'authentification basé sur les rôles

1.3 Périmètre Fonctionnel

L'application couvre les domaines fonctionnels suivants :

- Authentification et gestion des sessions utilisateur
- Recherche et filtrage de salles avec critères multiples
- Création, modification et annulation de réservations
- Gestion administrative des salles et équipements
- Gestion des comptes utilisateurs
- Tableau de bord et statistiques

2. Architecture Technique

2.1 Vue d'ensemble

Spacium est basé sur une architecture MVVM (Model-View-ViewModel) stricte, combinant les technologies Microsoft modernes pour une application de bureau robuste et maintenable.

2.2 Pile Technologique

Langage	C# 13 (.NET 10)
Framework UI	WPF (Windows Presentation Foundation)
UI Library	WPF-UI (Wpf.Ui 4.2.0)

ORM	Entity Framework Core 10.0.0
Base de Données	SQLite (Microsoft.EntityFrameworkCore.Sqlite)
MVVM Toolkit	CommunityToolkit.Mvvm 8.4.0
DI Container	Microsoft.Extensions.Hosting 10.0.1
Cible	Windows 10+ (net10.0-windows)

2.3 Pattern MVVM

L'architecture respecte strictement le pattern MVVM :

- Models : Entités métier (User, Salle, Reservation, Equipement, Creneau)
- Views : Pages et fenêtres XAML (ReservationPage, BookRoomPage, RoomsPage, etc.)
- ViewModels : Logique métier avec ObservableObject et RelayCommand
- Services : Couche de services (AuthService, DatabaseInitializerService)

2.4 Injection de Dépendances

L'application utilise le Generic Host de Microsoft.Extensions.Hosting comme conteneur IoC. Toutes les dépendances (DbContext, Services, ViewModels, Pages, Windows) sont enregistrées dans App.xaml.cs et injectées automatiquement.

3. Structure du Projet

3.1 Organisation des Dossiers

Dossier	Contenu	Description
Models/	User, Salle, Reservation, Equipement, Creneau	Entités de la base de données
Data/	ApplicationDbContext	Contexte Entity Framework Core
Services/	AuthService, DatabaseInitializerService, IAuthService	Services métier et infrastructure
ViewModels/Pages/	DashboardVM, BookRoomVM, ReservationVM, RoomsVM, GestionnaireVM, LoginVM	ViewModels des pages principales
ViewModels/Windows/	MainWindowVM, UserEditDialogVM	ViewModels des fenêtres
Views/Pages/	DashboardPage, BookRoomPage, ReservationPage, RoomsPage, GestionnairePage, LoginPage, SettingsPage	Pages de navigation

Views/Windows/	MainWindow, ReservationWindow, ReservationEditWindow, RoomEditWindow, EquipmentEditWindow, UserEditWindow, DialogWindow	Fenêtres de l'application
Converters/	BoolToVisibilityConverter, StatusToBrushConverter, FunctionalStatusConverter, etc.	Convertisseurs XAML
Helpers/	EnumToBooleanConverter	Assistants de liaison XAML
Assets/	initialize_database.sql, MIGRATION_EN_COURS_STATUS.sql	Scripts SQL et ressources
Migrations/	InitialCreate	Migrations Entity Framework

4. Base de Données

4.1 Technologie

Spacium utilise SQLite comme base de données embarquée. Le fichier app.db est créé automatiquement dans le répertoire d'exécution de l'application. L'initialisation est gérée par DatabaseInitializerService qui exécute le script SQL initialize_database.sql intégré comme ressource embarquée.

4.2 Schéma de la Base de Données

La base de données comprend les tables suivantes :

Table	Colonnes Clés	Description
Users	id, username, email, password, role, nom	Comptes utilisateurs (User/Gestionnaire)
Salles	id, nom, description, capacite, type, etage, disponibilite	Salles disponibles à la réservation
Equipements	id, nom, description, type, estFonctionnel, salleId	Equipements associés aux salles (FK salleId)
Creneaux	id, debut, fin	Créneaux horaires prédéfinis (08h-18h par tranche de 2h)
Reservations	id, dateReservation, motif, statut, userId, salleId, creneauId, dateDebut, dateFin, heureDebut, heureFin	Réservations avec statuts dynamiques
Historique	id, action, dateAction, reservationId	Journal des actions sur réservations

4.3 Statuts des Réservations

Le cycle de vie d'une réservation suit les états suivants, mis à jour automatiquement à chaque chargement de la page :

Statut	Couleur	Condition
En attente	Jaune (#FFCC00)	Date/heure de début dans le futur
Confirmée	Bleu (#0078D4)	Réservation validée manuellement
En cours	Vert (#107C10)	Date/heure actuelle entre début et fin
Terminée	Gris (#666666)	Date/heure de fin dépassée
Annulée	Rouge (#D83B01)	Annulée manuellement par l'utilisateur

4.4 Vues SQL

Trois vues sont définies pour simplifier les requêtes complexes :

- v_reservations_completes : Jointure complète Reservations + Users + Salles + Creneaux avec calcul de durée
- v_salles_equipements : Salles avec comptage et liste des équipements fonctionnels
- v_statistiques_utilisateurs : Statistiques par utilisateur (total, confirmées, en attente, annulées, heures)

5. Système d'Authentification

5.1 Mécanisme

L'authentification est gérée par AuthService (singleton) implémentant IAuthService. Le service maintient l'état de l'utilisateur connecté et émet un événement UserChanged lors des connexions/déconnexions, permettant au menu principal de se mettre à jour dynamiquement.

5.2 Rôles et Permissions

Rôle	Accès et Fonctionnalités
User	Tableau de bord, Réserver une salle, Mes Réservations (ses propres réservations uniquement), Paramètres
Gestionnaire	Tableau de bord, Réserver une salle, Toutes les Réservations, Gestion des Salles, Gestion des Utilisateurs, Paramètres

5.3 Comptes par Défaut

Identifiant	Mot de passe / Rôle
user1	user1 — Rôle : User
gestionnaire	gestionnaire — Rôle : Gestionnaire

6. Fonctionnalités Détaillées

6.1 Réservation de Salle

Processus de réservation

1. L'utilisateur accède à « Réserver une salle »
2. Il filtre les salles par nom, capacité minimale, étage ou équipements
3. Il clique sur « Réserver » sur la carte de salle de son choix
4. La fenêtre ReservationWindow s'ouvre avec les champs :
   - Motif (obligatoire)
   - Date de début et date de fin (DatePicker)
   - Créneau horaire ou horaires personnalisés
5. L'application vérifie les conflits pour chaque jour de la période
6. En cas de conflit, une boîte de dialogue avertit l'utilisateur avec option de forcer la réservation
7. La réservation est sauvegardée en base avec le statut « Confirmée »

6.2 Gestion des Réservations

La page Reservations offre les fonctionnalités suivantes :

- Filtrage par statut, salle et motif
- Modification des réservations « En attente » ou « Confirmée » uniquement
- Annulation avec confirmation (interdit pour « Terminée »)
- Suppression avec confirmation (toujours disponible)
- Mise à jour automatique des statuts au chargement de la page
- Les boutons sont désactivés intelligemment selon le statut

6.3 Gestion des Salles (Gestionnaire)

Interface de gestion complète accessible uniquement au rôle Gestionnaire :

- Ajout d'une nouvelle salle avec : nom, description, capacité, type, étage, disponibilité
- Modification des salles existantes
- Suppression de salles
- Gestion des équipements par salle : ajout, édition, suppression
- Chaque équipement possède : nom, type, description, état fonctionnel
- Recherche par nom, description, type ou équipement

6.4 Gestion des Utilisateurs (Gestionnaire)

- Liste de tous les utilisateurs avec informations complètes
- Ajout d'un nouvel utilisateur (nom, email, username, password, rôle)
- Modification des profils utilisateurs
- Suppression de comptes

- Recherche par nom, email ou username

7. Interface Utilisateur

7.1 Design System

Spacium utilise la bibliothèque WPF-UI (Wpf.Ui 4.2.0) qui implémente le design Fluent Design System de Microsoft. Le thème Light est appliqué par défaut avec possibilité de basculer en Dark depuis les paramètres.

7.2 Navigation

La navigation est gérée par NavigationView de WPF-UI avec un volet latéral gauche. Le menu s'adapte dynamiquement selon le rôle de l'utilisateur connecté. Le service INavigationService permet la navigation programmatique entre les pages.

7.3 Convertisseurs XAML

Convertisseur	Usage
BoolToVisibilityConverter	Visibilité selon booléen
BoolToVisibilityConverterInverse	Visibilité inverse selon booléen
NullToVisibilityConverter	Masque si valeur null
BoolToAvailabilityStringConverter	Affiche « ✓ Disponible » ou « ✗ Non disponible »
StringToVisibilityConverter	Masque si chaîne vide
EmptyCollectionToVisibilityConverter	Affiche message si collection vide
FunctionalStatusConverter	Affiche « ✓ Fonctionnel » ou « ✗ En panne »
StatusToBrushConverter	Couleur de fond du badge selon statut
StatusToForegroundBrushConverter	Couleur du texte du badge (contraste)
StatusToEnabledConverter	Désactive les boutons selon le statut

7.4 Fenêtres et Dialogues

Fenêtre	Rôle
MainWindow	Fenêtre principale avec NavigationView
ReservationWindow	Formulaire de nouvelle réservation
ReservationEditWindow	Modification d'une réservation existante
RoomEditWindow	Ajout/édition d'une salle et ses équipements
EquipmentEditWindow	Ajout/édition d'un équipement

UserEditWindow	Ajout/édition d'un utilisateur
DialogWindow	Boîte de dialogue générique (Info/Warning/Error/Confirmation)

8. Données Initiales

8.1 Salles Pré-configurées

Salle	Capacité	Type
Amphithéâtre A101	150 pers.	Amphithéâtre (Étage 1)
Salle de Cours B201	35 pers.	Salle de cours (Étage 2)
Salle de Cours B202	35 pers.	Salle de cours (Étage 2)
Laboratoire C301	25 pers.	Laboratoire informatique (Étage 3)
Laboratoire C302	25 pers.	Laboratoire scientifique (Étage 3)
Salle de Séminaire D102	50 pers.	Séminaire (Étage 1)
Salle de Réunion E103	15 pers.	Réunion (Étage 1)
Salle de Travail Collaboratif F104	20 pers.	Collaboration (Étage 1)

8.2 Créneaux Horaires

15 créneaux horaires sont pré-définis, correspondant aux tranches de 2 heures de 08h00 à 18h00 (08-10h, 10-12h, 12-14h, 14-16h, 16-18h), répétés sur 3 groupes.

9. Installation et Déploiement

9.1 Pré-requis

- Windows 10 ou supérieur (x64)
- .NET 10.0 Runtime (Windows Desktop Runtime)
- Environ 50 MB d'espace disque

9.2 Initialisation de la Base de Données

Au premier lancement, si le fichier app.db n'existe pas, DatabaseInitializerService crée automatiquement la base depuis le script SQL embarqué (initialize_database.sql). Aucune action manuelle n'est requise.

En cas de problème de schéma (mise à jour), il suffit de supprimer le fichier app.db dans le répertoire d'exécution et de relancer l'application.

9.3 Compilation

- Ouvrir ReserV7.slnx dans Visual Studio 2022 ou supérieur
- Restaurer les packages NuGet
- Compiler en mode Release (net10.0-windows)
- L'exécutable généré se trouve dans bin/Release/net10.0-windows/

10. Dépendances NuGet

Package	Version	Usage
WPF-UI	4.2.0	Bibliothèque UI Fluent Design pour WPF
WPF-UI.DependencyInjection	4.2.0	Intégration DI pour WPF-UI
Microsoft.Extensions.Hosting	10.0.1	Generic Host, DI Container, Configuration
CommunityToolkit.Mvvm	8.4.0	ObservableObject, RelayCommand, source generators
Microsoft.EntityFrameworkCore	10.0.0	ORM pour accès aux données
Microsoft.EntityFrameworkCore.Sqlite	10.0.0	Fournisseur SQLite pour EF Core

11. Points Clés de l'Architecture

11.1 Gestion des Statuts

La méthode UpdateReservationStatuses() est appelée à chaque chargement de la page Réservations. Elle compare la date/heure actuelle avec les dates de début et de fin de chaque réservation pour mettre à jour automatiquement les statuts (En attente / En cours / Terminée), à l'exception des réservations annulées qui conservent leur statut.

11.2 Détection de Conflits

Lors de la création d'une réservation, l'application génère toutes les dates entre dateDebut et dateFin, et vérifie pour chacune l'absence de chevauchement d'horaire avec des réservations existantes pour la même salle. En cas de conflit, l'utilisateur est averti et peut choisir de continuer ou d'annuler.

11.3 Validation

Chaque formulaire implémente une validation côté client avant toute écriture en base :

- Motif de réservation obligatoire
- Dates et horaires cohérents (fin > début)
- Format des horaires HH:mm validé
- Capacité de salle supérieure à 0
- Nom d'utilisateur unique à la création

11.4 Séparation des Responsabilités

Le projet applique rigoureusement la séparation des responsabilités : les ViewModels ne référencent pas directement les vues, les services sont injectés via les constructeurs, et les commandes (RelayCommand) assurent la liaison entre l'interface et la logique métier.

12. Glossaire

Terme	Définition
MVVM	Model-View-ViewModel : pattern d'architecture séparant la logique métier de l'UI
WPF	Windows Presentation Foundation : framework d'interface graphique .NET
EF Core	Entity Framework Core : ORM permettant l'accès aux données via des objets C#
SQLite	Système de gestion de base de données embarquée, sans serveur
DI / IoC	Dependency Injection / Inversion of Control : patron d'injection de dépendances
Créneau	Tranche horaire prédéfinie pour la réservation (ex: 08h00-10h00)
Fluent Design	Système de design Microsoft implémenté par WPF-UI
Observable	Propriété qui notifie l'interface lors de ses changements (INotifyPropertyChanged)
RelayCommand	Implémentation d'ICommand liant une action de l'UI à une méthode ViewModel